

Correção do RGB (Scroll Lock)

Ubuntu 26.04 + GNOME (Wayland) — Teclado Gamer Genérico USB

Este documento guarda tudo o que foi feito para resolver o problema do RGB do teclado que era ativado pela tecla Scroll Lock e que não funcionava no Ubuntu 26.04. Guarde este arquivo: se o RGB parar de funcionar de novo (por exemplo depois de trocar o teclado ou atualizar o sistema), volte aqui para reaplicar ou entender a solução.

O problema

O kit de teclado gamer (genérico, semi mecânico) usa um truque comum em teclados baratos: o efeito de RGB é acionado pelo próprio firmware do teclado quando o computador liga o LED padrão de Scroll Lock. Isso funciona normalmente no Windows e também no modo texto puro do Linux (TTY), porque o kernel liga esse LED diretamente. Só que o GNOME, quando roda em Wayland (o padrão no Ubuntu 26.04, já que o GNOME removeu o suporte a Xorg a partir da versão 49), não sincroniza esse LED automaticamente com o hardware. Resultado: apertar Scroll Lock não fazia nada.

A solução

Criamos um pequeno programa em Python que fica rodando sempre em segundo plano, escutando o teclado diretamente. Toda vez que a tecla Scroll Lock é apertada, o programa escreve manualmente no arquivo do sistema que controla aquele LED, ligando ou desligando o RGB. Esse programa foi configurado como um serviço do systemd, o que significa que ele liga sozinho sempre que o computador é ligado, sem precisar fazer nada manualmente.

Arquivos criados no sistema:

- Script Python: `/usr/local/bin/scrolllock-rgb.py`
- Serviço systemd: `/etc/systemd/system/scrolllock-rgb.service`

1. O Script Python — scrolllock-rgb.py

Este é o programa que fica rodando o tempo todo e faz o RGB funcionar. Abaixo está o código completo, dividido em partes, com a explicação de cada trecho logo abaixo dele.

1.1 — Importações

```
#!/usr/bin/env python3
import evdev
from evdev import ecodes
import glob
import time
```

A primeira linha diz ao Linux que esse arquivo deve ser executado com Python 3. **evdev** é a biblioteca que permite ler eventos de teclado/mouse diretamente do sistema (foi instalada com o comando `sudo apt install python3-evdev`). **ecodes** traz os códigos padronizados de cada tecla (ex: o código da tecla Scroll Lock). **glob** serve para buscar arquivos por um padrão de nome (usado para achar o LED automaticamente). **time** é usado para pequenas pausas no meio do programa.

1.2 — Função que encontra o LED do Scroll Lock

```
def find_led_path():
    matches = glob.glob("/sys/class/leds/*scrolllock*")
    if matches:
        return matches[0] + "/brightness"
    return None
```

O Linux representa o LED de Scroll Lock como um arquivo dentro de `/sys/class/leds/`, mas o número exato (por exemplo `input2::scrolllock` ou `input24::scrolllock`) muda a cada vez que o computador liga, dependendo da ordem em que o Linux detecta os dispositivos USB. Por isso, em vez de escrever esse número fixo no código (o que quebrava o script depois de reiniciar), essa função procura, na hora em que o programa inicia, qualquer pasta que tenha "scrolllock" no nome e usa ela automaticamente.

1.3 — Função que encontra o teclado certo

```
def find_keyboard():
    for path in evdev.list_devices():
        dev = evdev.InputDevice(path)
        caps = dev.capabilities().get(ecodes.EV_KEY, [])
        if ecodes.KEY_SCROLLLOCK in caps and ecodes.KEY_A in caps:
            return dev
    return None
```

O computador pode ter vários dispositivos de entrada (teclado, mouse, botões multimídia, etc). Essa função passa por todos os dispositivos conectados e verifica qual deles consegue enviar tanto a tecla Scroll Lock quanto a tecla A — ou seja, qual é o teclado de verdade, e não algum dispositivo secundário (como o "Consumer Control" ou "System Control" que aparecem separados no mesmo teclado USB).

1.4 — Preparação antes de começar a escutar

```
LED_PATH = find_led_path()
device = find_keyboard()

if LED_PATH is None or device is None:
    raise SystemExit("Teclado ou LED não encontrado ainda")

state = 0
```

Aqui o programa chama as duas funções acima para descobrir o caminho do LED e qual é o teclado. Se por algum motivo nenhum dos dois for encontrado (por exemplo, se o script tentar rodar rápido demais, antes do Linux terminar de reconhecer o teclado no boot), o programa para com uma mensagem de erro — e o systemd (explicado na seção 2) vai tentar rodar ele de novo automaticamente até dar certo. A variável `state` guarda se o RGB está ligado (1) ou desligado (0) neste momento.

1.5 — Função que liga/desliga o LED

```
def set_led(value):
    with open(LED_PATH, "w") as f:
        f.write(str(value))
```

Essa função simplesmente abre o arquivo do LED que encontramos antes e escreve nele o número 0 (desligado) ou 1 (ligado). É exatamente o mesmo efeito de rodar manualmente o comando `echo 1 | sudo tee /sys/class/leds/.../brightness` que usamos lá no começo para testar.

1.6 — O loop principal (o "coração" do programa)

```
for event in device.read_loop():
    if event.type == ecodes.EV_KEY and event.code == ecodes.KEY_SCROLLLOCK and event.value == 1:
        state = 1 - state
        set_led(state)
    elif event.type == ecodes.EV_LED:
        time.sleep(0.05)
        set_led(state)
```

Essa é a parte que roda para sempre, esperando eventos do teclado. Ela faz duas coisas:

- **Quando você aperta Scroll Lock** (`event.value == 1` significa "tecla pressionada", não "solta"): o programa inverte o estado guardado (se estava 0 vira 1, se estava 1 vira 0) e manda esse novo valor para o LED, ligando ou desligando o RGB.
- **Quando o GNOME muda algum LED** (`EV_LED`, que acontece quando você aperta Caps Lock ou Num Lock): o teclado recebe do GNOME um pacote com o estado dos 3 LEDs juntos, e isso apagava sem querer o nosso Scroll Lock. Por isso, sempre que isso acontece, o script espera 0,05 segundos (tempo para o GNOME terminar de escrever) e força o LED de volta para o estado que estava guardado.

2. O Serviço systemd — scrolllock-rgb.service

O systemd é o sistema do Ubuntu responsável por ligar programas automaticamente, inclusive durante o boot. Criamos um "serviço" para que o script Python de cima rode sozinho, sem precisar abrir terminal nenhum, toda vez que o computador ligar.

```
[Unit]
Description=Sincroniza LED do Scroll Lock com RGB do teclado
After=multi-user.target

[Service]
ExecStart=/usr/bin/python3 /usr/local/bin/scrolllock-rgb.py
Restart=always
RestartSec=2
User=root

[Install]
WantedBy=multi-user.target
```

[Unit]	Seção com informações gerais sobre o serviço.
Description=...	Um nome descritivo, que aparece quando você consulta o status do serviço.
After=multi-user.target	Diz ao systemd para só tentar iniciar este serviço depois que o sistema já estiver com o ambiente multiusuário básico pronto (evita tentar rodar cedo demais).
[Service]	Seção que descreve como o programa deve ser executado.
ExecStart=...	O comando exato que o systemd deve rodar: chama o Python 3 apontando para o nosso script.
Restart=always	Se o programa fechar ou der erro por qualquer motivo, o systemd deve iniciar ele de novo automaticamente.
RestartSec=2	Espera 2 segundos antes de tentar reiniciar. Isso foi importante porque, logo depois do boot, o teclado às vezes ainda não tinha sido reconhecido pelo Linux — o script falhava, esperava 2 segundos, tentava de novo, até dar certo.
User=root	Roda o programa como administrador (root), porque escrever no arquivo do LED exige essa permissão.
[Install]	Seção que diz como o serviço se conecta ao processo de boot do sistema.
WantedBy=multi-user.target	Faz o serviço ligar junto com o sistema operacional, sempre que o computador é ligado.

3. Todos os comandos usados no terminal

Abaixo está cada comando usado durante a resolução do problema, na ordem em que foram executados, separados um por um com a explicação do que cada um faz.

Comando 1

```
echo $XDG_SESSION_TYPE
```

Mostra se a sessão gráfica atual está rodando em Wayland ou em Xorg (X11). Usamos para descobrir que o sistema estava em Wayland, o que explicava o LED não sincronizar.

Comando 2

```
lsusb
```

Lista todos os dispositivos conectados via USB, incluindo o teclado, com o nome do fabricante. Ajuda a identificar o hardware conectado.

Comando 3

```
ls /usr/share/xsessions/
```

Lista as sessões gráficas em modo Xorg instaladas no sistema. Deu erro de "pasta não existe" porque, a partir do Ubuntu 25.10/26.04, o GNOME removeu totalmente o suporte a Xorg.

Comando 4

```
sudo apt update
```

Atualiza a lista de pacotes disponíveis para instalação, para garantir que os comandos de instalação seguintes peguem as versões mais recentes.

Comando 5

```
sudo apt install xserver-xorg gnome-session
```

Tentativa de instalar o servidor gráfico Xorg e a sessão do GNOME. Foi um passo do caminho que se mostrou sem saída, já que o GNOME 49 não oferece mais sessão Xorg.

Comando 6

```
dpkg -L gnome-session | grep xsessions
```

Lista todos os arquivos instalados pelo pacote gnome-session e filtra apenas os relacionados a sessões gráficas. Usamos para confirmar que o pacote não trazia nenhuma sessão Xorg, só Wayland.

Comando 7

```
ls /sys/class/leds/ | grep -i scroll
```

Procura, dentro da pasta de LEDs do sistema, qualquer um com "scroll" no nome. É assim que encontramos o arquivo exato que controla o LED de Scroll Lock (e o RGB junto).

Comando 8

```
echo 1 | sudo tee /sys/class/leds/input24::scrolllock/brightness
```

Escreve o valor 1 diretamente no arquivo do LED, ligando o RGB manualmente. Foi o teste que confirmou que o problema podia ser resolvido escrevendo direto nesse arquivo, sem depender do GNOME.

Comando 9

```
cat /proc/bus/input/devices | grep -A 5 -i keyboard
```

Lista os dispositivos de entrada (teclado, mouse etc) reconhecidos pelo Linux, mostrando nome, identificação e qual arquivo ("event") corresponde a cada um. Usamos para descobrir qual event pertence ao teclado de verdade.

Comando 10

```
sudo apt install python3-evdev
```

Instala a biblioteca Python que permite ler eventos de teclado diretamente do sistema. É usada dentro do script scrolllock-rgb.py.

Comando 11

```
sudo mkdir -p /usr/local/bin && sudo tee /usr/local/bin/scrolllock-rgb.py > /dev/null << 'EOF' ... EOF
```

Cria a pasta /usr/local/bin (caso não exista) e, em seguida, cria o arquivo do script Python com todo o conteúdo entre as marcações EOF, sem precisar abrir nenhum editor de texto interativo.

Comando 12

```
sudo chmod +x /usr/local/bin/scrolllock-rgb.py
```

Dá permissão de execução ao script, permitindo rodar ele diretamente como um programa.

Comando 13

```
sudo python3 /usr/local/bin/scrolllock-rgb.py
```

Executa o script manualmente no terminal, para testar se ele funciona antes de deixá-lo automático. Enquanto roda, o terminal fica ocupado esperando eventos do teclado (Ctrl+C para parar).

Comando 14

```
sudo tee /etc/systemd/system/scrolllock-rgb.service > /dev/null << 'EOF' ... EOF
```

Cria o arquivo de configuração do serviço systemd, que ensina o Ubuntu a rodar o script sozinho, inclusive nos boots seguintes.

Comando 15

```
sudo systemctl daemon-reload
```

Avisa o systemd que arquivos de serviço novos ou modificados foram criados, para que ele releia as configurações.

Comando 16

```
sudo systemctl enable scrolllock-rgb.service
```

Marca o serviço para iniciar automaticamente toda vez que o computador ligar.

Comando 17

```
sudo systemctl start scrolllock-rgb.service
```

Inicia o serviço imediatamente, sem precisar reiniciar o computador.

Comando 18

```
sudo systemctl restart scrolllock-rgb.service
```

Reinicia o serviço, útil depois de alterar o script, para que a versão nova entre em uso.

Comando 19

```
systemctl status scrolllock-rgb.service --no-pager
```

Mostra se o serviço está ativo (active/running), há quanto tempo, e as últimas linhas de log — sem travar o terminal num modo de leitura (--no-pager).

Comando 20

```
systemctl is-enabled scrolllock-rgb.service
```

Confirma rapidamente se o serviço está marcado para iniciar no boot (resposta esperada: "enabled").

Comando 21

```
journalctl -u scrolllock-rgb.service --no-pager
```

Mostra o histórico completo de mensagens (logs) gerado por esse serviço específico. Útil para investigar erros caso o RGB pare de funcionar novamente no futuro.

Comando 22

```
sudo reboot
```

Reinicia o computador por completo. Foi usado para testar se o serviço realmente inicia sozinho, sem qualquer comando manual, depois de ligar o PC.

Fim do documento. Guarde este PDF — se o RGB parar de responder ao Scroll Lock no futuro, comece revendo a seção 3 pelos comandos de status e log (systemctl status e journalctl).